

Transaction Isolation Levels

id: sd-001 | **Sub-topic:** databases | **Confidence:** (*pending*)/5 **First logged:** 2026-08-19 | **Last updated:** 2026-08-19 | **Sessions:** 1

Source Material (what I brought in)

From the system-design PDF. User's summary:

"Database isolation levels (which are used to allow a transaction to execute as if there are no other concurrently running transactions). There are 4 levels ([lowest] read uncommitted, read committed, repeatable read, serializable [highest]). This is done using Shared Locks and Exclusive Locks and something called MVCC (Multi Version Consistency Control). There was a small example using repeatable read to explain how MVCC works but it was ambiguous and not really helpful."

Requested: full detail on the topic and every term in it, assuming no background beyond basic CS.

Correction Pass

Three things to fix, one of them load-bearing:

1. **"allow a transaction to execute as if there are no other concurrently running transactions"** — that is the definition of **SERIALIZABLE** only. It is not what isolation levels *are*; it is what the strongest one gives you. Isolation levels exist precisely to let you **buy less than that** in exchange for throughput. Defining the category by its top member hides the entire point of the category.
2. **"MVCC (Multi Version Consistency Control)"** — Multi-Version **Concurrency** Control. Not a nitpick: it is a concurrency-control mechanism, sitting in the same slot as locking. It is not a consistency model, and it does not by itself decide an isolation level.
3. **"This is done using Shared Locks and Exclusive Locks and MVCC"** — these are two *competing* implementation families, not one combined mechanism. The SQL standard mandates **neither**. It defines the levels purely by which anomalies they forbid, and any engine may hit those targets however it likes. This is why "repeatable read" means materially different things in MySQL and PostgreSQL.

Correct as stated: the four names and their ordering, and that all four are attempts at the same underlying problem.

What It Is

An isolation level is a **contract about which concurrency anomalies the database is allowed to expose to you**, chosen per transaction, trading correctness guarantees for throughput.

It is a dial, not a feature. Turning it up costs latency and aborts; turning it down costs you invariants you may not realize you depend on.

The Problem It Solves

Foundations, from the bottom

A **transaction** is a group of reads and writes the database treats as a single unit. The classic guarantee bundle is ACID: **A**tomicity (all-or-nothing), **C**onsistency (constraints hold), **I**solation (this topic), **D**urability (survives a crash).

Notation used throughout, from Berenson et al. (1995) — it's compact and worth learning:

```
r1[x]    transaction 1 reads item x
w1[x]    transaction 1 writes item x
c1       transaction 1 commits
a1       transaction 1 aborts
r1[P]    transaction 1 reads the set of rows matching predicate P
```

Why not just run them one at a time?

Serial execution is trivially correct. It's also unusably slow: a transaction spends most of its life waiting on disk and network, and a serial database would leave the CPU idle through all of it while every other client queues.

So databases **interleave** operations from concurrent transactions. That is the source of every problem below.

The correctness bar

Serializability: the outcome of running transactions concurrently must equal the outcome of running them one at a time in *some* serial order.

Note "some". Not a specific order — you don't get to pick, and the database doesn't promise the order matches wall-clock start times. Any serial order counts as correct.

The anomalies (what you're buying protection from)

These are the actual vocabulary. Levels are defined by which of these they forbid.

#	Name	History	What it means
P0	Dirty Write	$w1[x] \dots w2[x] \dots (c1 \text{ or } a1)$	T2 overwrites data T1 wrote but hasn't committed. Rolling back T1 now can't restore a sane state.
P1	Dirty Read	$w1[x] \dots r2[x] \dots (c1 \text{ or } a1)$	T2 reads T1's uncommitted write. If T1 aborts, T2 acted on data that never existed.
P2	Non-repeatable / Fuzzy Read	$r1[x] \dots w2[x] \dots c2 \dots r1[x]$	T1 reads x twice, gets different values, because T2 committed in between.
P3	Phantom	$r1[P] \dots w2[y \text{ in } P] \dots (c1 \text{ or } a1)$	T1 re-runs a <i>query</i> , gets a different <i>set of rows</i> — someone inserted/deleted a row matching the predicate.

#	Name	History	What it means
P4	Lost Update	$r1[x] \dots w2[x] \dots w1[x] \dots c1$	Classic read-modify-write clobber. T1's update silently erases T2's.
A5A	Read Skew	$r1[x] \dots w2[x] \dots w2[y] \dots c2 \dots r1[y]$	T1 reads x, then y, and gets a mutually inconsistent <i>pair</i> — each valid at a different instant.
A5B	Write Skew	$r1[x] \dots r2[y] \dots w1[y] \dots w2[x] \dots (c1 \text{ and } c2)$	Two transactions read an overlapping set, write <i>disjoint</i> rows, and jointly break a constraint neither broke alone.

P2 vs P3 is the distinction people get wrong. P2 is a *row's value* changing under you. P3 is *set membership* changing under you — a row appearing or vanishing from a query result. You can hold a lock on every row you read (killing P2) and still get P3, because you cannot lock a row that does not exist yet. That single fact is why "repeatable read" and "serializable" are separate levels at all.

The standard's table

Level	Dirty read	Non-repeatable read	Phantom	Serialization anomaly
Read uncommitted	Possible	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Possible	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

Read this as a **permission** table, not a behavior table. It says what an engine *may* expose, not what it *will*. An engine is free to be stricter than the level requires — and in practice they all are, in different places, which is the source of most portability bugs.

How It Works

Family A: Locking (pessimistic)

Shared lock (S) = read lock. Many transactions may hold S on the same item at once. Blocks writers.

Exclusive lock (X) = write lock. Exactly one holder. Blocks everyone.

	S held	X held
want S	✓ compatible	✗ block
want X	✗ block	✗ block

Locks alone don't give serializability. **Two-Phase Locking (2PL)** does: every transaction has a *growing* phase (may only acquire) and a *shrinking* phase (may only release). Once you release anything, you may never

acquire again. **Strict 2PL** holds all locks until commit — that's what real engines do, because releasing early lets others read data you might still roll back.

Here's the part that makes the whole taxonomy click. **The four levels are just "how long do you hold read locks, and do you lock things that don't exist yet":**

Level	Write (X) locks	Read (S) locks
Read uncommitted	Held to commit	Not taken at all
Read committed	Held to commit	Taken, released immediately after the read
Repeatable read	Held to commit	Held until commit
Serializable	Held to commit	Held until commit, plus locks on predicates/ranges

Walk it downward and every anomaly falls out mechanically:

- No S locks on read → you can read uncommitted data → **dirty read**.
- S released immediately → the row can change before you read it again → **non-repeatable read**.
- S held to commit → the rows you read are frozen. But a *new* row matching your query can still be inserted, because you locked rows, and you cannot lock the absence of a row → **phantom**.
- Lock the *predicate* (or the index range / the gaps between keys) → the insert itself is blocked → phantoms gone → **serializable**.

Cost: blocking. A long-running read holds S locks and stalls every writer touching those rows. Plus deadlocks, which the engine resolves by picking a victim and aborting it.

Family B: MVCC (multi-version)

The core move: **never overwrite a row in place**. Every write creates a new *version* of the row, tagged with the transaction that created it. Old versions stay until no one can still need them.

Conceptually each version carries:

```
xmin  the transaction id that created this version
xmax  the transaction id that deleted/superseded it (empty if current)
```

A transaction takes a **snapshot**: effectively "the set of transactions that had committed at this instant". A version is visible to you if its **xmin** is committed-as-of-your-snapshot and its **xmax** is not.

The payoff, and the reason MVCC won:

Readers never block writers. Writers never block readers.

A twenty-minute analytical scan reads the versions that were current when it started, while writes proceed at full speed creating newer ones. Under pure locking that scan would freeze the table.

Writers still block writers. You still need X locks (or *first-committer- wins*: at commit, if anyone committed a write to a row you wrote since your snapshot, you abort) — otherwise you'd have dirty writes and lost updates.

Here the level maps to *when you take the snapshot*, not to lock duration:

Level under MVCC	Snapshot taken
Read committed	A fresh one per statement
Repeatable read / snapshot isolation	One, at transaction start, reused for every statement

That is the whole mechanism, and it's what your PDF's ambiguous example was reaching for. Concretely — `balance = 100`, and a concurrent transaction commits `balance = 500` between the two reads:

```

READ COMMITTED (new snapshot each statement)
T1: BEGIN
T1: SELECT balance FROM acct WHERE id=1;  -- snapshot #1 -> 100
      T2: UPDATE acct SET balance=500 WHERE id=1; COMMIT;
T1: SELECT balance FROM acct WHERE id=1;  -- snapshot #2 -> 500  ← changed!
T1: COMMIT
      Both reads were of committed data. Neither was a dirty read.
      The value still changed underneath T1. That is P2.

REPEATABLE READ (one snapshot, at BEGIN)
T1: BEGIN                                -- snapshot fixed here
T1: SELECT balance FROM acct WHERE id=1;  -- -> 100
      T2: UPDATE acct SET balance=500 WHERE id=1; COMMIT;
T1: SELECT balance FROM acct WHERE id=1;  -- -> 100  ← still the old version
T1: COMMIT
      T2's new version has xmin > T1's snapshot, so it is invisible to T1.
      The old version is still physically present. Nothing blocked.

```

Note what T1 sees is not "stale" in any sloppy sense — it is a genuine, complete, committed state of the database, just an earlier one. That's the distinction worth holding: MVCC gives you *a consistent past*, not *the uncertain present*.

The trap: snapshot isolation is not serializable

Freezing one snapshot for the whole transaction kills dirty reads, non-repeatable reads, **and phantoms for free** — a query re-run against a frozen instant cannot see an insert that happened after it. So SI looks like it must be serializable.

It isn't. **Write skew (A5B) survives**, and this is the single most important thing on this page.

MVCC detects conflicts only when two transactions **write the same row**. Write skew is two transactions writing *different* rows after reading an overlapping set. First-committer-wins never fires — there's no write-set overlap to detect.

The paper's canonical history H5 — `x = 50, y = 50`, invariant `x + y ≥ 0`:

```
H5:  r1[x=50] r1[y=50] r2[x=50] r2[y=50] w1[y=-40] w2[x=-40] c1 c2
```

```
T1 reads x=50, y=50. Sum 100. "I can safely set y = -40; sum stays 10." ✓
```

T2 reads x=50, y=50. Sum 100. "I can safely set x = -40; sum stays 10." 
 Both commit. They wrote different rows, so nothing conflicts.
 Final state: x = -40, y = -40. Sum = -80. Invariant violated.

There is no serial order producing this: run T1 fully first and T2 reads y=-40 and refuses. Each transaction was individually correct against a genuinely consistent snapshot. The *pair* is not.

Berenson et al. state the relationship formally:

Remark 9. REPEATABLE READ »« Snapshot Isolation.

»« means **incomparable** — neither is stronger. SI forbids phantoms that standard RR permits; RR forbids write skew that SI permits. This is routinely misreported as "SI is stronger than RR". It isn't; it's sideways.

Reaching real serializability

1. **Strict 2PL + predicate locks** — correct, blocks heavily.
2. **SSI (Serializable Snapshot Isolation)** — PostgreSQL 9.1+. Keeps SI's non-blocking reads, adds tracking of read/write dependencies, and aborts a transaction when a dangerous cycle is detected. PostgreSQL's predicate locks *"do not cause any blocking and therefore can not play any part in causing a deadlock"* — they exist only to flag dependencies. Cost: **false-positive aborts**. You must have retry logic.
3. **Actual serial execution** — single-threaded (Redis, VoltDB). Correct by construction; scales only via partitioning.

Trade-offs

Choice	Buys you	Costs you	Use when
Read uncommitted	Marginally less bookkeeping	Dirty reads — you may act on data that gets rolled back	Almost never. PostgreSQL doesn't even implement it.
Read committed	No dirty reads; short/no read locks; high concurrency	P2, P3, lost updates, write skew	Default for most OLTP. Correct when each statement is self-contained.
Repeatable read / SI	Stable snapshot for the whole transaction; consistent multi-statement reads	Write skew; under MVCC, aborts on write-write conflict	Reports, multi-statement reads that must agree, read-modify-write on a <i>single</i> row.
Serializable	Every anomaly gone; you can reason serially	Throughput; blocking (2PL) or abort rate (SSI); mandatory retry loop	Cross-row invariants that must hold — balances, booking limits, on-call coverage.

Rule of thumb: the moment your correctness depends on a constraint spanning *multiple rows* that concurrent transactions could each satisfy alone, snapshot isolation is not enough. Go serializable, or take an

explicit lock (`SELECT ... FOR UPDATE`) on a row that both transactions must contend for — materializing the conflict MVCC couldn't see.

Numbers That Matter

No published latency figures are cited here — isolation-level overhead is dominated by workload contention, so vendor-neutral numbers would be misleading. The figures worth memorizing are structural, not performance:

- **4** standard levels; PostgreSQL implements **3** distinct ones (*"PostgreSQL's Read Uncommitted mode behaves like Read Committed"*).
- **SQLSTATE 40001** — serialization failure. PostgreSQL: *"It is important that an environment which uses this technique have a generalized way of handling serialization failures (which always return with an SQLSTATE value of '40001')"*. Your retry handler keys on this.
- **Read-only transactions never abort under SSI**: *"only updating transactions might need to be retried; read-only transactions will never have serialization conflicts."* Hence PostgreSQL's advice to *"declare transactions as READ ONLY when possible."*

In Real Systems

Engine	Default	Notable deviation from the standard
PostgreSQL	Read committed	No real read uncommitted. Repeatable read <i>is</i> snapshot isolation and forbids phantoms (stricter than the standard). Serializable = SSI, genuinely serializable.
MySQL / InnoDB	Repeatable read	Plain <code>SELECT</code> uses a consistent snapshot from the first read. <i>Locking</i> reads take gap / next-key locks , blocking inserts into scanned ranges — so RR blocks many phantoms too. Under read committed, <i>"gap locking is disabled"</i> , so <i>"phantom row problems may occur"</i> . Serializable is implemented by silently converting plain <code>SELECT</code> to <code>SELECT ... FOR SHARE</code> .
Oracle	Read committed	No true repeatable read. Its SERIALIZABLE is snapshot isolation — it does not prevent write skew, so it is not serializable despite the name.

Two portability landmines follow directly:

- MySQL and PostgreSQL both say "repeatable read" and give you different guarantees (MySQL adds gap locks on locking reads; PostgreSQL gives SI).
- Oracle's **SERIALIZABLE** and PostgreSQL's **SERIALIZABLE** are different isolation levels wearing the same word.

Common Misconceptions

- **"Isolation levels are implemented with locks."** Two families exist; MVCC is the dominant one in modern engines. The standard mandates neither.
- **"Repeatable read prevents phantoms."** Under the standard, no. Under PostgreSQL and MySQL, largely yes, by different mechanisms. The name tells you nothing — check your engine.

- **"Snapshot isolation is serializable."** No. Write skew survives, and it's the anomaly most likely to corrupt real business invariants.
- **"Serializable means transactions run in the order I submitted them."** It means the result matches *some* serial order. Not necessarily yours.
- **"Higher isolation = slower, uniformly."** Under SSI, reads don't block at all; the cost appears as *aborts* under write contention, not latency. A low-contention workload can run serializable near-free.
- **"MVCC means readers see stale data."** They see a consistent committed *past state*, which is a different and much stronger thing than stale.
- **"A serialization failure is a bug."** It's the designed signal to retry. Missing retry logic is the bug.

Worked Example

Hospital on-call: at least one doctor must be on call at all times. The textbook write skew, and it fails in production under a default PostgreSQL/Oracle configuration.

Table `doctors`: Alice `on_call = true`, Bob `on_call = true`. Both feel unwell and click "go off call" simultaneously.

<pre>-- T1 (Alice) BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT count(*) FROM doctors WHERE on_call = true; -- 2, so it's safe to leave UPDATE doctors SET on_call = false WHERE name = 'Alice'; COMMIT; ✓</pre>	<pre>-- T2 (Bob) BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT count(*) FROM doctors WHERE on_call = true; -- 2, so it's safe to leave UPDATE doctors SET on_call = false WHERE name = 'Bob'; COMMIT; ✓</pre>
---	---

Both succeed. **Zero doctors on call.** No dirty read, no non-repeatable read, no phantom, no write-write conflict — the two `UPDATE`s touch *different rows*, so first-committer-wins has nothing to fire on. Each transaction verified the invariant against a genuinely consistent snapshot and was individually correct.

Three fixes, in increasing order of cost:


```
-- 1. SERIALIZABLE – PostgreSQL's SSI sees the rw-dependency cycle
--    and aborts one with SQLSTATE 40001. Requires a retry loop.
BEGIN ISOLATION LEVEL SERIALIZABLE;

-- 2. Materialize the conflict: force both onto the same rows with an
--    explicit lock, converting invisible write skew into a visible
--    write-write conflict. Works at REPEATABLE READ and even READ COMMITTED.
SELECT count(*) FROM doctors WHERE on_call = true FOR UPDATE;

-- 3. Push it into the database as a constraint the engine enforces
--    (a trigger, or a materialized counter row with a CHECK), so no
--    application path can violate it regardless of isolation level.
```

Fix 2 is the one worth internalizing: **when a constraint spans rows, give the transactions a single row to fight over**. That's the general escape hatch from write skew without paying for full serializability.

Discussion Log

Session 1 — 2026-08-19

(cross-questioning in progress — to be filled in on close)

Open Questions

(to be filled in on close)

Connections

- [context/concept-bank.md](#) → [sysd-03](#) (CAP theorem) — isolation is the "C" of ACID, which is a different C from CAP's consistency; worth untangling when that topic comes up.
- Future: [system-design/consistency/](#) docs on consensus and distributed transactions — isolation is the single-node story, linearizability the distributed one.

Sources

- PDF: user's system-design PDF, isolation levels section
- [PostgreSQL 18 docs — Transaction Isolation](#) — level table, SI/SSI, predicate locks, 40001, read-only advice
- [Berenson, Bernstein, Gray, Melton, O'Neil, O'Neil — A Critique of ANSI SQL Isolation Levels \(SIGMOD 1995\)](#) — P0–P4/A5 formal definitions, snapshot isolation, first-committer-wins, H5 write skew, Remark 9
- [MySQL 8.4 docs — InnoDB Transaction Isolation Levels](#) — RR default, gap/next-key locks, RC gap-locking disabled
- [Oracle serializable is not serializable — dbi services](#) — Oracle's SERIALIZABLE is snapshot isolation